

Inhaltskonzept & Grundlagenrecherche zur Bachelorarbeit

LED-Produktkonfigurator – Wissenstransfer, Konzeption und
prototypische Umsetzung (configuLED, LANG AG)

Arbeitsunterlage – ausformulierter Fließtext je Gliederungspunkt
Stand: 31.07.2026

Inhaltsverzeichnis

- 1 Einleitung
 - 1.1 Problemstellung und Motivation
 - 1.2 Zielsetzung und Forschungsfrage
 - 1.3 Aufbau der Arbeit
- 2 Theoretische Grundlagen
 - 2.1 LED-Technik im Veranstaltungsbereich

2.2 Produktkonfiguratoren als interaktive Medienanwendungen

2.3 Wissenstransfer – von Expertenwissen zu nutzbaren Interfaces

2.4 KI-gestützte Softwareentwicklung

3 Analyse des Ist-Zustands

3.1–3.5 (Methodik als Fließtext, empirisch zu füllen)

4 Konzeption

4.1 Anforderungsdefinition

4.2 Informationsarchitektur und Nutzerführung

4.3 UX/UI-Konzept

4.4 Visualisierungskonzept

5 Prototypische Umsetzung

5.1 Technologie-Stack

5.2 KI-gestützte Entwicklung

5.3 Umsetzung des Prototyps

6 Evaluation

6.1 Methodik

6.2 Durchführung und Ergebnisse

6.3 Rückspiegelung an die Anforderungen

Hinweis: Kapitel 3 und 6.2 benötigen echte Erhebungsdaten (Interviews, Tests, Konkurrenzrecherche), die hier nicht erfunden, sondern als ausformulierter methodischer Text vorstrukturiert werden.

1 Einleitung

1.1 Problemstellung und Motivation

LED-Wände für den Veranstaltungsbereich sind hochgradig konfigurierbare Produkte: Pixelpitch, Modulmaße, Montageart, Indoor- oder Outdoor-Eignung, feste versus temporäre Installation, Kurvbarkeit und Traglastanforderungen für Traversen- und Rigging-Systeme ergeben zusammen einen kombinatorisch großen Lösungsraum. Dieses Wissen liegt in der Praxis überwiegend implizit bei Produktmanagement und Vertrieb der LANG AG vor, sodass eingehende Kundenanfragen bislang individuell und manuell geprüft werden müssen, bevor eine belastbare Konfiguration entstehen kann.

Daraus ergibt sich ein doppeltes Problem. Zum einen ist der bestehende Beratungsprozess personalgebunden, zeitintensiv und lässt sich nur schwer skalieren, da jede Anfrage individuelles Fachwissen erfordert. Zum anderen sind Kund:innen ohne tiefergehendes technisches Fachwissen, etwa Veranstalter:innen, die nur selten LED-Wände anmieten, kaum in der Lage, technisch valide Konfigurationen eigenständig zusammenzustellen. Damit steigt das Risiko fachlich inkorrektter Anfragen, etwa der unzulässigen Kombination aus Außeneinsatz und einem nicht ausreichend geschützten Modul oder der Überschreitung zulässiger Traglasten bei einer Hängung.

Der bei LANG AG bereits eingesetzte Konfigurator configuLED deckt diesen Bedarf, wie die Ist-Analyse in Kapitel 3 zeigen wird, nur unvollständig ab. Die Motivation dieser Arbeit besteht folglich darin, das bei LANG AG vorhandene Expertenwissen systematisch zu erfassen und in ein selbsterklärendes, für fachfremde Nutzer:innen bedienbares Interface zu überführen, wobei zusätzlich untersucht wird, wie sich der Einsatz KI-gestützter Entwicklungswerkzeuge auf eine realistische Umsetzung innerhalb des zeitlichen Rahmens einer Bachelorarbeit auswirkt.

An dieser Stelle sollten noch firmenspezifische, belastbare Fakten ergänzt werden, etwa die durchschnittliche Bearbeitungszeit einer Anfrage, die Anzahl der Supportanfragen pro Monat oder eine bekannte Fehlerquote. Das macht aus einer allgemein gehaltenen Problemstellung eine konkrete und im weiteren Verlauf der Arbeit überprüfbare Motivation.

1.2 Zielsetzung und Forschungsfrage

Die Arbeit verfolgt, wie für gestaltungsorientierte Design-Science-Arbeiten typisch, ein zweigeteiltes Ziel. Das Gestaltungsziel besteht in der Konzeption und prototypischen Umsetzung eines webbasierten LED-Konfigurators, der die Konfigurationslogik von

configuLED ablöst beziehungsweise sinnvoll erweitert. Das Erkenntnisziel besteht darin, zu verstehen, wie sich Expertenwissen methodisch in Konfigurationsregeln und Interface-Logik überführen lässt und welchen Effekt dies auf Usability und Konfigurationssicherheit hat.

Als zentrale Forschungsfrage bietet sich folgende Formulierung an:

„Wie muss ein webbasierter LED-Produktkonfigurator gestaltet sein, damit Nutzer:innen ohne spezifisches Fachwissen technisch valide Konfigurationen erstellen können, und welchen Beitrag leistet KI-gestützte Softwareentwicklung zur prototypischen Umsetzung eines solchen Systems?“

Aus dieser Leitfrage lassen sich sinnvollerweise zwei bis drei Unterforschungsfragen ableiten, die sich jeweils einem Kapitel der Arbeit zuordnen lassen. Die erste Unterfrage betrifft den Wissenstransfer und lautet sinngemäß, wie sich implizites Konfigurationswissen aus Vertrieb und Produktmanagement in explizite, prüfbare Regeln überführen lässt; sie wird in den Kapiteln 2.3 und 4.1 behandelt. Die zweite Unterfrage betrifft Informationsarchitektur und Nutzerführung und fragt, welche Struktur und Führung die wahrgenommene Komplexität für fachfremde Nutzer:innen reduziert; sie wird in den Kapiteln 4.2, 4.3 und 6 aufgegriffen. Die dritte Unterfrage betrifft den Entwicklungsprozess selbst und untersucht, wie der Einsatz agentischer KI-Coding-Werkzeuge den Entwicklungsprozess eines solchen Prototyps verändert; sie wird in Kapitel 5.2 beantwortet.

Als methodischer Rahmen für die gesamte Arbeit eignet sich Design Science Research nach Hevner et al. (2004) beziehungsweise Peffers et al. (2007). Dieser Ansatz rechtfertigt den iterativen Aufbau der Arbeit von der Analyse über die Konzeption zum Artefakt, dem Prototyp, bis hin zur abschließenden Evaluation.

1.3 Aufbau der Arbeit

Kapitel 2 legt die theoretischen Grundlagen zu LED-Technik, Produktkonfiguratoren, Wissenstransfer und KI-gestützter Softwareentwicklung, auf denen die weiteren Kapitel aufbauen. Kapitel 3 analysiert anschließend den Ist-Zustand bei LANG AG, den bestehenden Konfigurator configuLED, das Wettbewerbsumfeld sowie die relevante Zielgruppe. Darauf aufbauend leitet Kapitel 4 ein Konzept ab, das Anforderungsdefinition, Informationsarchitektur, UX/UI-Konzept und Visualisierungskonzept umfasst. Kapitel 5 beschreibt die prototypische Umsetzung einschließlich des Einsatzes KI-gestützter Entwicklungswerkzeuge. Kapitel 6 evaluiert den entstandenen Prototyp und spiegelt die

Ergebnisse gegen die in Kapitel 4 definierten Anforderungen zurück, um die in Kapitel 1.2 formulierte Forschungsfrage abschließend zu beantworten.

Dieser Abschnitt sollte erst final formuliert werden, wenn die Kapitelstruktur nicht mehr verändert wird, da er den tatsächlichen Aufbau eins zu eins wiedergeben muss.

2 Theoretische Grundlagen

2.1 LED-Technik im Veranstaltungsbereich

Dieser Abschnitt liefert das technische Vokabular, das in der Datenbank- und Konfigurationslogik des Prototyps in Kapitel 5 unmittelbar wiederauftaucht. Zentrale Kenngröße einer LED-Wand ist der Pixelpitch P , der den Abstand zweier benachbarter LED-Mittelpunkte in Millimetern bezeichnet. Aus ihm lässt sich die Auflösung eines einzelnen Panels sowie einer aus mehreren Modulen zusammengesetzten Wand ableiten:

$$\begin{aligned} \text{Panel-Auflösung: } R_x &= \text{Breite}_{\text{Panel}} [\text{mm}] / P & R_y &= \text{Höhe}_{\text{Panel}} [\text{mm}] / P \\ \text{Gesamtauflösung bei } n \text{ Modulen: } R_{x,\text{gesamt}} &= (n_x \cdot \text{Modulbreite} [\text{mm}]) / P \end{aligned}$$

Aus dem Auflösungsvermögen des menschlichen Auges, das etwa eine Bogenminute beträgt, lässt sich außerdem der physikalisch sinnvolle Mindestbetrachtungsabstand einer LED-Wand herleiten:

$$\begin{aligned} \text{MVD} [\text{mm}] &= P [\text{mm}] / \tan(1') \approx P [\text{mm}] \times 3438 \\ \Rightarrow \text{MVD} [\text{m}] &\approx P [\text{mm}] \times 3,4 \end{aligned}$$

In der Praxis wird in der Branche meist eine vereinfachte Faustformel verwendet, die etwa ein bis drei Meter Betrachtungsabstand je Millimeter Pixelpitch veranschlagt, wobei die untere Grenze die technisch noch nutzbare und die obere Grenze die komfortable Betrachtung markiert. Für die Bachelorarbeit bietet es sich an, beide Herleitungen, also die physikalisch exakte Berechnung und die vereinfachte Industrie-Faustformel, einander gegenüberzustellen und zu diskutieren, weshalb Hersteller in ihrer Kommunikation meist die konservativere Faustformel bevorzugen.

Neben der optischen Auflösung sind Leistungsaufnahme und Gewicht zentrale technische Kenngrößen, die im späteren Konfigurator geprüft werden müssen. Die Gesamtleistungsaufnahme einer Wand ergibt sich als Summe der Einzelleistungen aller Module, wobei die tatsächliche Durchschnittslast im Betrieb, abhängig vom dargestellten Content, üblicherweise nur etwa dreißig bis fünfzig Prozent der maximalen Leistung beträgt. Das Gesamtgewicht ergibt sich entsprechend als Produkt aus Modulanzahl und Einzelgewicht:

$$P_{\text{gesamt}} = \sum P_{\text{Modul},i} \quad G_{\text{gesamt}} = n \cdot g_{\text{Modul}}$$

Das Gesamtgewicht bildet die zentrale Eingangsgröße für die statische Prüfung der eingesetzten Traversen- und Hängesysteme, wie sie im Prototyp in der Datei `liam_truss.py` umgesetzt ist. An dieser Stelle lohnt sich in der Theorie ein kurzer Exkurs zu den Grundlagen von Traversensystemen, insbesondere zu zulässigen Punktlasten und Spannweiten, bevor dieser Zusammenhang in Kapitel 5.3 konkret am Prototyp beschrieben wird.

Ein weiterer für diese Arbeit relevanter technischer Aspekt ist die Krümmung modularer LED-Wände. Über mechanische Scharniere zwischen benachbarten Panels lässt sich eine Wand konvex oder konkav krümmen. Geometrisch lässt sich eine solche gekrümmte Wand aus n Panels der Breite w , die jeweils um denselben Knickwinkel θ zueinander geneigt sind, als Sehnenviel Polygon eines Kreisbogens modellieren:

Radius des Bogens: $R = w / (2 \cdot \sin(\theta/2))$

Gesamtöffnungswinkel: $\theta_{\text{gesamt}} = n \cdot \theta$

Näherung der Bogenlänge bei kleinem θ : $s \approx n \cdot w$

Diese Herleitung gehört fachlich in das Theoriekapitel, während in den Kapiteln 4.4 und 5.3 die bewusste Vereinfachung für die tatsächliche Visualisierung, also eine zweidimensionale Illusion anstelle einer exakten dreidimensionalen Geometrie, als eigenständige Designentscheidung mit Begründung dargestellt werden sollte. Diese Vereinfachung ist kein fachlicher Abstrich, sondern eine bewusste Abwägung zwischen Implementierungsaufwand und wahrgenommenem Nutzen für die Anwender:innen.

2.2 Produktkonfiguratoren als interaktive Medienanwendungen

Produktkonfiguratoren lassen sich als klassische Ausprägung von Mass Customization im Sinne von Piller und Möslin einordnen: Sie erlauben es, aus einer begrenzten Menge modularer Komponenten kundenindividuelle Varianten zusammenzustellen, ohne dass jede denkbare Variante einzeln entwickelt oder vorab bevorratet werden müsste. Bei k unabhängigen Konfigurationsdimensionen, etwa Pixelpitch, Montageart, Standort und Krümmung, mit jeweils n_i möglichen Ausprägungen ergibt sich die Anzahl theoretisch möglicher Varianten als Produkt:

$$N_{\text{Varianten}} = \prod_{i=1}^k n_i$$

Diese kombinatorische Explosion begründet unmittelbar, weshalb ein rein listenbasierter Ansatz, bei dem alle Varianten vorab einzeln angelegt werden, nicht skaliert, und weshalb

stattdessen ein regelbasiertes System notwendig ist. Ein solches Konfigurationsproblem lässt sich formal als Tripel aus Variablen, Domänen und Constraints beschreiben: Die Variablen X entsprechen den Konfigurationsdimensionen wie Montageart, Pixelpitch oder Standort, die Domänen D den jeweils zulässigen Werten je Variable, und die Constraints C den Bedingungen zwischen diesen Variablen, etwa der Regel, dass ein Außeneinsatz mindestens die Schutzklasse IP65 voraussetzt. Diese Formalisierung bildet die theoretische Brücke zur tatsächlichen Implementierung, denn im Prototyp entsprechen die Constraints unmittelbar den booleschen Spalten der Artikeltabelle, etwa `is_indoor_capable`, `is_outdoor_capable`, `is_fixed_capable` und `is_temporary_capable`, sowie der darauf aufbauenden Filterlogik in `server.py`.

Ergänzend lässt sich ein Produktkonfigurator auch als Decision-Support-System beziehungsweise als Instrument des Guided Selling einordnen: Anders als ein bloßes Eingabeformular führt er aktiv durch den Entscheidungsprozess und reduziert dabei schrittweise die Komplexität für die Nutzer:innen.

2.3 Wissenstransfer – von Expertenwissen zu nutzbaren Interfaces

Eine zentrale theoretische Grundlage dieses Abschnitts ist die Unterscheidung zwischen implizitem, also nicht direkt artikulierbarem Wissen, und explizitem, dokumentierbarem Wissen, wie sie im SECI-Modell nach Nonaka und Takeuchi beschrieben wird. In der Phase der Socialization wird Wissen zwischen Personen ausgetauscht, etwa im Rahmen eines Interviews mit dem Produktmanagement der LANG AG. In der anschließenden Phase der Externalization wird dieses zunächst implizite Wissen in eine explizite Form, also in Regeln oder Dokumentation, überführt. In der Phase der Combination wird das nun explizite Wissen systematisiert, beispielsweise in Entscheidungstabellen oder im Datenbankschema des Konfigurators. In der abschließenden Phase der Internalization wird das entstandene System genutzt, wodurch bei den Anwender:innen wiederum neues implizites Verständnis entsteht, das den Kreislauf erneut in Gang setzen kann.

Praktisch lässt sich Expertenwissen besonders gut über Entscheidungstabellen und sogenannte Business Rules formalisieren, die dem einfachen Schema folgen, dass unter einer bestimmten Bedingung eine bestimmte Aktion ausgelöst wird. Dieses Schema lässt sich unmittelbar auf die in Kapitel 2.2 eingeführte Constraint-Logik sowie auf die konkrete Filterlogik in Kapitel 5.3 übertragen. Als Referenzliteratur eignen sich hierzu einschlägige Werke zum Wissensmanagement, etwa von Probst, Raub und Romhardt, sowie Literatur zur Requirements-Elicitation bei der Befragung von Fachexpert:innen, etwa das Standardwerk von Pohl zum Requirements Engineering.

2.4 KI-gestützte Softwareentwicklung

Für diesen Abschnitt ist zunächst eine Abgrenzung sinnvoll zwischen klassischen Autocomplete-Assistenten, wie sie frühe Versionen von GitHub Copilot darstellten, und agentischen KI-Coding-Systemen wie Claude Code, die selbstständig Dateien lesen, Änderungen vornehmen, Kommandozeilenwerkzeuge ausführen und iterativ auf Rückmeldungen reagieren können. Für die theoretische Einordnung sind insbesondere drei Konzepte relevant. Prompt Engineering beschreibt die Kommunikationsform zwischen Entwickler:in und Sprachmodell, über die Aufgaben präzisiert und iterativ verfeinert werden. Human-in-the-loop-Review bezeichnet den Umstand, dass jede von einem KI-Modell generierte Änderung weiterhin durch die Entwickler:in geprüft und verantwortet werden muss. Schließlich sind die Grenzen solcher Systeme zu benennen, insbesondere die Gefahr der Halluzination, also plausibel klingender, aber inhaltlich falscher Aussagen oder Code, sowie die daraus resultierende Notwendigkeit ausreichender Testabdeckung und eines fundierten Codeverständnisses trotz KI-Unterstützung.

Zu Produktivitätseffekten von KI-Coding-Assistenten existiert bereits Forschung, unter anderem im Umfeld von GitHub und Microsoft Research zu Copilot. Die konkreten Studien und Zahlen sollten selbstständig recherchiert und zitiert werden, da an dieser Stelle bewusst auf nicht verifizierbare Studienergebnisse oder Prozentangaben verzichtet wird.

3 Analyse des Ist-Zustands

Dieses Kapitel erfordert reale Erhebungsdaten, etwa Prozessdokumentationen, Konkurrenzrecherche und Interviews. Nachfolgend wird deshalb ausschließlich das methodische Vorgehen als Fließtext beschrieben.

3.1 Die LANG AG und ihr Produktmanagement-Prozess

Dieser Abschnitt sollte mit einem kurzen Unternehmensporträt beginnen, das Größe, Marktsegment und Produktportfolio der LANG AG umreißt. Im Anschluss empfiehlt sich eine Darstellung des bestehenden Ist-Prozesses in Form eines Flowcharts oder einer BPMN-Notation, die den typischen Ablauf von der eingehenden Anfrage über die fachliche Beratung und die Konfiguration bis hin zum fertigen Angebot abbildet. Aus dieser Prozessdarstellung lässt sich anschließend eine Liste konkreter Pain Points ableiten, etwa auffällig zeitintensiver Prozessschritte, wiederkehrender Fehlerquellen oder Medienbrüche zwischen den beteiligten Systemen, die im weiteren Verlauf der Arbeit als Ausgangspunkt für die Anforderungsdefinition in Kapitel 4.1 dienen.

3.2 configuLED – Analyse des bestehenden Konfigurators

Für die Analyse des bestehenden Konfigurators bietet sich eine heuristische Evaluation an, die die zehn Usability-Heuristiken nach Nielsen als Prüfraster verwendet und so systematisch Schwachstellen der bestehenden Lösung aufdeckt. Ergänzend empfiehlt sich eine Feature-Matrix, die die im bestehenden System vorhandenen Funktionen den in Kapitel 4 definierten Anforderungen gegenüberstellt und auf diese Weise sichtbar macht, an welchen Stellen der neue Prototyp über configuLED hinausgehen muss.

3.3 Wettbewerbsanalyse: LED-Konfiguratoren am Markt

Für den Vergleich mit am Markt verfügbaren LED-Konfiguratoren bietet sich ein Kriterienkatalog an, der unter anderem die Konfigurationstiefe, also die Anzahl konfigurierbarer Dimensionen, die Art der Visualisierung von keiner Darstellung über zweidimensionale bis hin zu dreidimensionaler Live-Vorschau, den Umfang der Regelprüfung von fehlender Validierung bis zu einer vollständigen Regel-Engine, das Vorhandensein einer automatisierten Preis- oder Angebotsausgabe sowie die Mobile-Fähigkeit umfasst. Eine solche Vergleichsmatrix lässt sich tabellarisch darstellen:

Kriterium	Mögliche Ausprägung
Konfigurationstiefe	Anzahl konfigurierbarer Dimensionen
Visualisierung	keine / 2D / 3D / Live-Preview
Regelprüfung	keine Validierung / einfache Constraints / vollständige Regel-Engine
Preis-/Angebotsausgabe	ja/nein, automatisiert?
Mobile-Fähigkeit	responsive ja/nein

Die konkreten Wettbewerbsprodukte und deren tatsächlicher Funktionsumfang müssen recherchiert werden, etwa über Herstellerwebsites und öffentlich zugängliche Produktdemos. Diese Recherche kann bei Bedarf durch eine gezielte Websuche unterstützt werden, sollte aber nicht aus dem Gedächtnis behauptet werden.

3.4 Zielgruppenanalyse

Für die Zielgruppenanalyse bietet sich die Persona-Methodik an, wobei sich aus der Branchenkenntnis heraus bereits einige plausible Rollen als Ausgangshypothese formulieren lassen, die durch die Kundenbefragung in Kapitel 3.5 verifiziert werden sollten. Eventtechniker:innen zeichnen sich durch ein hohes technisches Fachwissen aus und benötigen im Konfigurator vor allem detaillierte Kontrolle über einzelne technische Parameter. Verleiher:innen und Bühnenbauer:innen verfügen über ein mittleres Fachwissen und legen ihren Fokus stärker auf schnelle und zugleich korrekte Angebote als auf technische Detailtiefe. Architekt:innen und Planer:innen bringen häufig nur geringes LED-spezifisches Fachwissen mit und interessieren sich in erster Linie für das visuelle Ergebnis sowie die grundsätzliche Machbarkeit einer Installation. Systemintegrator:innen schließlich verfügen über hohes Fachwissen und benötigen insbesondere Exportfunktionen sowie technische Datenblätter, um die Konfiguration in eigene Planungsunterlagen übernehmen zu können.

3.5 Kundenbefragung zur Konfigurationslogik

Für die Kundenbefragung bietet sich ein Fragebogen mit fünfstufigen Likert-Skalen an, dessen Stichprobengröße sich sinnvollerweise an der vorhandenen Kundenkontaktiliste der LANG AG orientiert und entsprechend begründet werden sollte. Die Auswertung erfolgt zunächst deskriptiv über Mittelwert und Median je Einzelitem. Sofern mehrere Items ein gemeinsames Konstrukt messen sollen, etwa die wahrgenommene

Verständlichkeit der Konfigurationslogik, empfiehlt sich zusätzlich eine Prüfung der internen Konsistenz über Cronbachs Alpha:

$$\alpha = (k / (k-1)) \cdot (1 - \Sigma \sigma_i^2 / \sigma_{\text{total}}^2)$$

4 Konzeption

4.1 Anforderungsdefinition

Für die Anforderungsdefinition empfiehlt sich zunächst eine klare Trennung zwischen funktionalen und nicht-funktionalen Anforderungen. Die Priorisierung der so gesammelten Anforderungen lässt sich anschließend über die MoSCoW-Methode vornehmen, die zwischen zwingend notwendigen, wünschenswerten, optionalen und bewusst ausgeschlossenen Anforderungen unterscheidet. Die einzelnen Anforderungen selbst lassen sich sinnvoll als User Stories formulieren, die dem Muster folgen, dass eine bestimmte Rolle ein bestimmtes Ziel verfolgt, um einen bestimmten Nutzen zu erzielen. Jede so formulierte Anforderung sollte zudem eine eindeutige Kennung erhalten, etwa in der Form FA-01 für eine funktionale Anforderung, da diese Kennungen später in Kapitel 6.3 für die Requirements-Traceability-Matrix benötigt werden.

4.2 Informationsarchitektur und Nutzerführung

Zur Entwicklung der Informationsarchitektur eignen sich Methoden wie Card Sorting zur Kategorisierung der einzelnen Konfigurationsschritte, die Erstellung einer Sitemap sowie die Modellierung konkreter User-Flow-Diagramme. Als leitendes Gestaltungsprinzip bietet sich Progressive Disclosure an, bei dem nicht alle Konfigurationsoptionen gleichzeitig angezeigt werden, sondern schrittweise entsprechend den bereits getroffenen Nutzerentscheidungen freigeschaltet werden. Eine quantitative Begründung für dieses Vorgehen liefert das Hick'sche Gesetz, das den Zusammenhang zwischen der Anzahl gleichzeitig angebotener Auswahlmöglichkeiten und der benötigten Entscheidungszeit beschreibt:

$$RT = a + b \cdot \log_2(n + 1)$$

n = Anzahl gleichzeitig angebotener Auswahlmöglichkeiten; a, b = empirische Konstanten

Diese Formel liefert eine belastbare Begründung dafür, Konfigurationsschritte in mehrere, jeweils überschaubare Stufen zu gliedern, anstatt eine große Optionsliste auf einmal zu präsentieren. Sie lässt sich unmittelbar auf die im Prototyp bereits angelegte Reihenfolge von Standort über Einsatzart bis hin zu Produktserie und Modell anwenden, wie sie in der Filterlogik des Endpunkts `api/products` umgesetzt ist.

4.3 UX/UI-Konzept

Als Gestaltungsraster für das UX/UI-Konzept eignen sich erneut die zehn Usability-Heuristiken nach Nielsen sowie die klassischen Gestaltgesetze der Nähe, Ähnlichkeit und Kontinuität, mit deren Hilfe sich zusammengehörige Konfigurationsoptionen visuell gruppieren lassen. Vor der eigentlichen Umsetzung sollte dabei stets ein Wireframing beziehungsweise ein kleines Design System als methodischer Zwischenschritt stehen. Sofern die Arbeit auch Aspekte der Barrierefreiheit behandelt, lässt sich die Zugänglichkeit farblicher Gestaltung über das WCAG-Kontrastverhältnis quantitativ prüfen:

Kontrastverhältnis = $(L_1 + 0,05) / (L_2 + 0,05)$, mit $L_1 > L_2$ (relative Luminanz)
Mindestanforderung für Normaltext: 4,5 : 1

4.4 Visualisierungskonzept

Das Visualisierungskonzept knüpft unmittelbar an die in Kapitel 2.1 hergeleiteten geometrischen Zusammenhänge an. Zentral ist die Abwägung zwischen einer geometrisch exakten dreidimensionalen Darstellung und einer vereinfachten zweidimensionalen Illusion der LED-Wand, insbesondere bei der Darstellung der Krümmungsfunktion. Für dieses Projekt fällt die begründete Designentscheidung zugunsten einer einfachen zweidimensionalen Illusion, da diese ausreichend und zugleich deutlich performanter ist, solange keine konkreten Referenzbilder oder Kundenanforderungen eine exaktere Geometrie verlangen. Die in Kapitel 2.1 hergeleiteten Radius- und Winkelformeln bleiben dabei die fachliche Grundlage der Transformation, werden jedoch bewusst vereinfacht in eine zweidimensionale Canvas- beziehungsweise SVG-Darstellung übersetzt, anstatt eine vollständige dreidimensionale Rendering-Engine zu implementieren. Dieser Trade-off zwischen Implementierungsaufwand, Rendering-Performance und dem wahrgenommenen Mehrwert für die Nutzer:innen sollte in der Arbeit explizit als bewusste Entscheidung mit entsprechender Begründung dargestellt werden, anstatt als bloße Vereinfachung erwähnt zu werden.

5 Prototypische Umsetzung

5.1 Technologie-Stack

Der im Rahmen dieser Arbeit entwickelte Prototyp basiert, wie sich am aktuellen Stand des Repositories ablesen lässt, auf einem vergleichsweise schlanken und pragmatischen Technologie-Stack. Das Backend ist in Python mit dem Microframework Flask umgesetzt und stellt seine Funktionalität über REST-artige JSON-Endpunkte bereit, etwa über `api/products` zur Produktabfrage, `api/mounting-types` für die verfügbaren Montagearten und `api/product-series` für die Serienauswahl. Die Datenhaltung erfolgt über eine SQLite-Datenbank namens `konfigurator.db`, deren Schema und Stammdaten über mehrere serienspezifische SQL-Setup-Skripte gepflegt werden, unter anderem für die WP-, RX-, MV- und WT-Serie sowie für Aura Flex. Dieser Aufbau realisiert eine datengetriebene statt einer hartkodierte Konfigurationslogik. Das Frontend besteht aus serverseitig gerendertem HTML und CSS mit Vanilla-JavaScript in der Datei `templates/index.html`, ohne den Einsatz eines zusätzlichen Frontend-Frameworks. Insgesamt lässt sich der Aufbau als klassische Drei-Schichten-Architektur aus Präsentation, Anwendungslogik und Datenhaltung einordnen, die sich in der Arbeit gut anhand eines einfachen Architekturdiagramms zwischen Client, Flask-Server und SQLite-Datenbank veranschaulichen lässt.

5.2 KI-gestützte Entwicklung

Dieser Abschnitt lässt sich als reflektierte Prozessbeschreibung des tatsächlichen Vorgehens in diesem Projekt formulieren, da während der Entwicklung durchgängig das agentische KI-Coding-Werkzeug Claude Code eingesetzt wurde. Dabei sollte zunächst dargestellt werden, welche Teilaufgaben per Prompt an das Werkzeug delegiert wurden, etwa Erweiterungen des Datenbankschemas, die Implementierung einzelner API-Endpunkte oder die Umsetzung der Filterlogik. Anschließend sollte der Review- und Korrekturprozess beschrieben werden, also inwiefern die vom Werkzeug erzeugten Vorschläge unverändert übernommen oder gezielt angepasst wurden. Ein konkretes und belegbares Beispiel für die Notwendigkeit menschlicher Steuerung liefert die Entwicklung der Kurvungs-Visualisierung: Hier wurde bewusst so lange auf konkrete Referenzbilder gewartet, bis eine Entscheidung getroffen wurde, anstatt eine vom Sprachmodell vorgeschlagene, geometrisch möglicherweise korrekte, aber ungetestete Lösung unreflektiert zu übernehmen. Dieses Vorgehen belegt, dass die fachliche Zielsetzung und das tatsächliche Nutzerbedürfnis stets Vorrang vor rein technischer Eleganz haben

müssen. Damit ordnet sich dieser Abschnitt methodisch als Prozess- und Erfahrungsbericht ein, nicht als reine Werkzeugbeschreibung, und liefert zugleich das inhaltliche Antwortmaterial für die dritte Unterforschungsfrage aus Kapitel 1.2.

5.3 Umsetzung des Prototyps

Die Umsetzung des Prototyps lässt sich anhand seiner tatsächlich implementierten Kernfunktionen beschreiben. Zentral ist zunächst die Filterlogik nach Standort, also Innen- oder Außeneinsatz, und Nutzungsart, also feste oder temporäre Installation: Jede Bedingung filtert dabei unabhängig auf ihrer eigenen Datenbankspalte, sodass ein sogenanntes Allrounder-Produkt, das sowohl als `is_indoor_capable` als auch als `is_outdoor_capable` markiert ist, automatisch beide Filter erfüllt. Auf diese Weise entsteht das gewünschte logische Oder zwischen den Standortoptionen, ohne dass sich Innen- und Außenfilter gegenseitig ausschließen. Dies ist zugleich ein anschauliches Beispiel dafür, wie die in Kapitel 2.2 eingeführten Constraints konkret in SQL-Bedingungen übersetzt wurden. Ein weiteres zentrales Element ist die Serien- und Modellauswahl, bei der eine einzelne Produktserie mehrere Modelle beziehungsweise Pixelpitches anbieten kann, was einer Eins-zu-Viele- anstelle einer starren Eins-zu-Eins-Beziehung entspricht. Darüber hinaus wurde die in Kapitel 2.1 hergeleitete Gewichts- und Traglastberechnung in Form einer mechanischen Traversenberechnung in der Datei `liam_truss.py` umgesetzt, und die in Kapitel 4.4 begründete vereinfachte zweidimensionale Darstellung wurde als eigenständiges Kurvungs-Feature implementiert. Zur weiteren Veranschaulichung eignet sich ein Entity-Relationship-Diagramm des Datenbankschemas, das den in Kapitel 2.3 beschriebenen Wissenstransfer sichtbar macht, indem es zeigt, wie ursprünglich implizite Expertenregeln in konkrete Tabellenspalten und Constraints überführt wurden.

Berechnungslogik der Bauteile je Montageart (Stücklisten-Formeln)

Ein zentraler und fachlich anspruchsvoller Teil des Prototyps ist die automatische Ermittlung der Stückliste für die jeweils gewählte Montageart. Jede der bei LANG AG vorhandenen Montagesysteme, Stacking, Hanging und Wandadapter, folgt dabei einer eigenen, aus dem Expertenwissen des Produktmanagements abgeleiteten Berechnungsvorschrift. Im Folgenden werden die tatsächlich implementierten Formeln dokumentiert, wobei B stets die Wandbreite in Cabinets, H die Wandhöhe in Cabinets und s die produktabhängige Stacker-Spannweite bezeichnet.

Basisalgorithmus: 3-2-1-Breitenverteilung

Nahezu alle Stacking- und Hanging-Systeme greifen auf denselben Basisalgorithmus zur Zerlegung der Wandbreite in Baugruppen der Breite 3, 2 oder 1 Cabinet zurück. Es

handelt sich um einen gierigen Algorithmus, der von links beginnend stets das größtmögliche Bauteil setzt, mit einer gezielten Ausnahme für den Sonderfall einer verbleibenden Restbreite von genau vier Cabinets, bei dem zwei 2er- anstelle eines 3er- und eines vermeidbaren 1er-Bauteils gesetzt werden:

```
rest ← B; b3 ← 0; b2 ← 0; b1 ← 0
solange rest > 0:
  wenn rest = 4: b2 ← b2 + 2; rest ← 0
  sonst wenn rest ≥ 3: b3 ← b3 + 1; rest ← rest - 3
  sonst wenn rest = 2: b2 ← b2 + 1; rest ← 0
  sonst: b1 ← b1 + 1; rest ← 0
```

Im Krümmungsmodus, also bei konkaver oder konvexer Krümmung der Wand, entfällt diese Optimierung vollständig, da an jeder Fuge unabhängig geknickt werden können muss. In diesem Fall wird stattdessen jede einzelne Cabinet-Spalte zu einem eigenen 1er-Bauteil:

Krümmungsmodus aktiv: $(b_3, b_2, b_1) = (0, 0, B)$

Standard-Stacking (Aura-Konzept)

Für das Standard-Stacking-System, dem Grundmodell für AURA, ARUNA, LUGH, ENKI, VENUS, JUPITER und weitere Serien, ergeben sich Footbeams, Stacker und Clamps wie folgt:

```
Footbeams F = B im Krümmungsmodus, sonst  $\lfloor B / 2 \rfloor$ 
Single Feet = 2 (fest, je eines an beiden Wandenden)
Stacker S = F ·  $\lfloor H / s \rfloor$  (s = 2 im Standardfall, s = 3 bei ENKI/VENUS/JUPITER)
Clamps = 2 · S (bzw. bei aktivem Krümmungsmodus oder B = 1: 0 normale Clamps, stattdessen
Curving-Clamp = B · ( $\lfloor H/s \rfloor$  · s))
```

AURA FLEX (abweichende Basement-Logik)

AURA FLEX verzichtet bewusst auf die 3-2-1-Verteilung und setzt stattdessen an jeder Fuge zwischen zwei benachbarten Cabinets ein einheitliches, 0,5 Cabinet breites Basement-Bauteil:

$\text{Basements}_{\text{Flex}} = \max(B - 1, 0)$

Footbeams und Stacker folgen unverändert der generischen Formel $F = \lfloor B/2 \rfloor$ und $S = F \cdot \lfloor H/s \rfloor$.

AR-Stacking (1:1-Footbeam-Variante)

Für die AR3.9-Produktfamilie ist das Footbeam-Raster nicht auf zwei, sondern auf jede einzelne Cabinet-Spalte bezogen, wodurch auch der Stacker abweichend berechnet wird:

$$\text{Footbeams } F = B \quad (1:1 \text{ zum Cabinet-Raster statt } \lceil B/2 \rceil)$$

$$\text{Stacker } S = F \cdot H$$

NoBase-System (Übergangsstück statt Basement)

Das NoBase-System verzichtet vollständig auf Basements. Stattdessen trägt für die untersten s Cabinet-Reihen ein eigenständiges Übergangsstück, bevor ab Reihe $s + 1$ das reguläre Stacker-System fortgesetzt wird:

$$\text{NoBase-Footbeams } FB = \lceil B / 2 \rceil$$

$$\text{NoBase-Stacker } N = FB, \text{ falls } H \geq s, \text{ sonst } 0$$

$$\text{restliche Reihen } R = \max(0, H - s); \text{ Stacker-Gruppen } G = \lceil R / s \rceil$$

$$\text{Stacker (regulär, oberhalb) } S = FB \cdot G$$

$$\text{NoBase-Clamps} = 3 \cdot N \text{ (bei } s = 2 \text{) bzw. } s \cdot N \text{ und zusätzlich 1 Standard-Clamp je}$$

$$\text{Instanz (bei } s = 3 \text{)}$$

$$\text{Standard-Clamps} = 2 \cdot S$$

Wandadapter

Wandadapter sitzen an den Schnittpunkten der vertikalen und horizontalen Cabinet-Fugen. Vertikal wird jede zweite Fuge gesichert, mit einer zusätzlichen Randbefestigung bei ungerader Wandbreite, horizontal jede tatsächlich vorhandene Fuge zwischen zwei übereinanderliegenden Cabinet-Reihen:

$$\text{vertikale Fugen mit Adapter } V = \lceil B / 2 \rceil$$

$$\text{horizontale Fugen mit Adapter } K = \max(0, H - 1)$$

$$\text{Wandadapter gesamt} = V \cdot K$$

Hanging-Systeme

Beim generischen Hanging-Truss-System, das unter anderem AURA, ARUNA, LUGH und LUNA gemeinsam nutzen, entspricht jedes 3-2-1-Modul zugleich genau einem Aufhängungspunkt; Footbeams, Stacker und Clamps entfallen vollständig, da die Wand nicht steht, sondern hängt:

$$\text{Aufhängungspunkte } P = b_3 + b_2 + b_1$$

$$\text{Ösen je Punkt} = 1, \text{ falls } H \leq \text{Grenzwert}_1 \text{ (produktspezifisch); sonst } 2, \text{ falls } H >$$

Grenzwert₂; sonst statisch nicht umsetzbar

Zusatzösen gesamt = P, falls Ösen je Punkt = 2, sonst 0

Drei Sondersysteme weichen davon ab. Vanish-Hanging und AR-Hanging benötigen keine weitere Stückliste jenseits der Hanging Bar selbst:

Vanish-Hanging: Hanging Bar = B (H ohne Einfluss)

AR-Hanging: Hanging Bar in 3-2-1-Verteilung (b_3, b_2, b_1), H ebenfalls ohne Einfluss

Vanish-Stacking bildet dagegen ein vollständig eigenständiges, geschlossenes Bausystem aus exakt sieben Bauteilen:

Stacking Bar = B Foot Beam = B Unterster Stacker = B

Stacker = $(H - 1) \cdot B$ Verbinder = $H \cdot B$

Splint = Klammer = $2 \cdot (\text{Stacking Bar} + \text{Foot Beam} + \text{Verbinder}) = 2B \cdot (H + 2)$

Ballastberechnung

Anders als die übrigen Bauteile lässt sich der benötigte Ballast nicht aus einer geschlossenen Formel ableiten, sondern basiert auf einer produktspezifischen Datenblatt-Tabelle (kg pro Stacker-Achse in Abhängigkeit von der Wandhöhe). Die Berechnung selbst ist gleichwohl trivial:

Ballast gesamt [kg] = Ballast pro Achse(H, Produkt) · Anzahl Stacker-Achsen

wobei eine Stacker-Achse einem Footbeam- beziehungsweise NoBase-Footbeam-Ankerpunkt entspricht. Dies ist ein gutes Beispiel dafür, dass nicht jedes Bauteil der Konfigurationslogik algorithmisch herleitbar ist, sondern Teile davon zwingend auf verifizierten Herstellerdatenblättern beruhen müssen.

LIAM-Traversensystem (vollständiges Gitter-Modell)

Das komplexeste Bauteilmodell des Prototyps betrifft den LIAM-Hanging-Modus, der als eigenständiges Gitter aus B vertikalen Traversen-Strings und mehreren horizontalen Ebenen modelliert wird. Jede einzelne Linie, vertikal wie horizontal, wird zunächst über eine Greedy-Längenoptimierung in 2-Meter- und 1-Meter-Module zerlegt:

für eine Linie der Länge L [m]:

Truss-2m-Module = $\lfloor L / 2 \rfloor$ Truss-1m-Module = $L \bmod 2$

Verbinder = $\max((\text{Truss-2m} + \text{Truss-1m}) - 1, 0)$

Die Anzahl der horizontalen Ebenen ergibt sich aus einem Raster, das vom oberen Rand aus abwärts zählt und die unterste Ebene stets exakt auf den Boden kappt, wobei Indoor ein 2-Meter- und Outdoor ein 1-Meter-Raster verwendet:

$$\text{Anzahl Ebenen } L_{\text{Ebenen}} = \lceil H / \text{Raster} \rceil \quad \text{Raster} = 2 \text{ m (Indoor) bzw. } 1 \text{ m (Outdoor)}$$

Im Indoor-Grundmodell, bei dem jede Ebene demselben 2-Meter-Brückenmuster folgt, entstehen Corner-Knoten ausschließlich an echten Schnittpunkten von vertikalem und horizontalem Modulende:

Corner-Spalten C = Anzahl Brückenpositionen je Ebene

$$\text{Corner-Knoten gesamt} = L_{\text{Ebenen}} \cdot C$$

Conn = Anzahl ungeradzahlgiger Spalten (1, 3, 5, ...) zzgl. 1, falls B geradzahlig ist

Bolzen: $2 \times \text{Bold-150}$ an jedem inneren, nicht-bodennahen Corner (echtes Vier-Wege-Kreuz), sonst $2 \times \text{Bold-100}$

$$\text{Nuten} = 4 \cdot \text{Corner-Knoten} + 2 \cdot \text{Conn} + (\text{Bold-100} + \text{Bold-150})$$

Das Outdoor-Modell verwendet dieselben Grundformeln, ersetzt jedoch das einheitliche Brückenmuster durch ein von oben nach unten strikt alternierendes Zeilenschema aus 1-Meter- und 2-Meter-Reihen, wobei die unterste Reihe unabhängig vom Alternationsrhythmus stets als 2-Meter-Reihe ausgeführt wird; nur auf den 2-Meter-Reihen entstehen dabei echte Corner-Knoten, während alle übrigen Berührungspunkte als zusätzliche Clamp-Bauteile geführt werden. Sofern das Bauteil zusätzlich gekurvt wird, kommt rein additiv ein Winkel-Bracket hinzu:

$$\text{Curving-Bracket} = (B - 1) \cdot H \quad (1x \text{ je gebogener vertikaler Fuge und Cabinet-Reihe})$$

6 Evaluation

6.1 Methodik

Für die Evaluation des Prototyps bietet sich ein Usability-Test-Design an, das ein Think-Aloud-Protokoll, klar definierte Testaufgaben, die Erfolgsquote je Aufgabe sowie eine Zeitmessung umfasst. Zur standardisierten Messung der wahrgenommenen Benutzerfreundlichkeit eignet sich zusätzlich die System Usability Scale, kurz SUS, die aus zehn Items auf einer fünfstufigen Likert-Skala besteht:

ungerade Items (1, 3, 5, 7, 9): Beitrag = Antwort – 1

gerade Items (2, 4, 6, 8, 10): Beitrag = 5 – Antwort

SUS-Score = (Summe aller Beiträge) $\times$ 2,5 $\rightarrow$ Skala von 0 bis 100

Werte über 68 gelten nach Bangor, Kortum und Miller als überdurchschnittlich und lassen sich zusätzlich über Sauros Grading-Skala von A bis F einordnen.

6.2 Durchführung und Ergebnisse

Dieser Abschnitt erfordert eine reale Testdurchführung mit echten Proband:innen und lässt sich daher an dieser Stelle nicht vorwegnehmen. Methodisch sinnvoll ist eine deskriptive Auswertung über Mittelwert und Standardabweichung, gegebenenfalls grafisch aufbereitet als Boxplot, sowie, sofern möglich, ein direkter Vergleich zwischen dem bestehenden Konfigurator configuLED und dem neuen Prototyp anhand identischer Testaufgaben.

6.3 Rückspiegelung an die Anforderungen

Den Abschluss der Evaluation bildet eine Requirements-Traceability-Matrix, in der jede in Kapitel 4.1 vergebene Anforderungs-Kennung dem jeweiligen Testergebnis gegenübergestellt und als erfüllt, teilweise erfüllt oder nicht erfüllt bewertet wird. Auf diese Weise schließt sich der durch Design Science Research vorgegebene Kreislauf aus Analyse, Konzeption, Umsetzung und Evaluation, und die in Kapitel 1.2 formulierte Forschungsfrage lässt sich direkt und nachvollziehbar anhand der so entstandenen Ergebnisse beantworten.

Offene Punkte – nicht erfindbar, sondern zu erheben

Für die Kapitel 3.1 bis 3.5 werden noch reale Prozess-, Wettbewerbs- und Befragungsdaten der LANG AG benötigt, die in diesem Dokument bewusst nicht vorweggenommen wurden. Für Kapitel 6.2 wird entsprechend die tatsächliche Durchführung eines Usability-Tests mit realen Messwerten benötigt. Und für Kapitel 2.4 sollten konkrete Literaturquellen zur Produktivitätsforschung im Bereich KI-gestützter Softwareentwicklung noch eigenständig geprüft und korrekt zitiert werden.